Yesterday 10:06 PM

You said:

Implementation …
File

https://piazza.com/ucsd/fall2026/ece285deepgenerativemodels/resources
https://drive.google.com/drive/folders/1mfsZijoWXvxFi53T6QImRUrY6Impealj
https://piazza.com/class/muwwwlp9vfi76w/#

I'd like your help completing Homework 1 for my Deep Generative Models class. I have very little background in DGNs, only the ML experience you can infer from my academic records / ML research projects / overall background, and I am not caught up on the lectures. I'll be starting the homework tomorrow morning, and I'd like to be done in at most 3 hours. Please create a study guide, suggest what to focus on from the course materials I've shared, and guide me through completing the homework, since the deadline is about 24 hours away. You don't have to give me all the answers; just guide me so I can study and learn as much as possible from the class content. Please also estimate the time it'll take to train the VAE on the chest X-ray dataset so that I can determine whether I can finish by tomorrow at midnight or if I'll have to use the extra submission dates. Please explain the hyperparameters, the Inception score, the FID score, and any other important topics.
…

Show more

Worked for 13m 3s

ChatGPT said:

Finishing by tomorrow night looks plausible if you have a working GPU. Finishing everything within three hours is an ambitious target, mainly because data setup, debugging, evaluation, and the report all have to fit alongside training.

I recommend one small VAE, a short timed pilot, and a single documented training run. You can learn the essential ideas while completing that experiment.

I created a guide with the schedule, video timestamps, explanations, code snippets, debugging checks, and a report outline:

Start with pages 1–2; consult the other sections as you work. Below is the main guidance so you can also use this conversation directly.

1. What you actually need to complete

Your one-page handout asks you to train a VAE on the linked chest X-ray dataset, generate images, and submit **code plus a two-page report in NeurIPS format** to Gradescope. It suggests including:

- Hyperparameter settings.
- Training-loss curves.
- Inception Score and FID.
- Examples of generated images.

I would include all four. The handout does **not** prescribe an architecture, image resolution, latent dimension, epoch count, or minimum score. Those are choices you can make and explain.

Source: mv47c1gud5n1ka.pdf

I could inspect the public Piazza resources and shared Drive folder, but **the syllabus and private announcements require sign-in**. Therefore, I could not verify the exact deadline, additional submission dates, or their penalties. Check those first tomorrow; the plan below uses your stated tomorrow-night deadline.

2. What to watch—and where to start

The supplied **Implementation of VAE** recording is approximately **85 minutes long**. It contains substantial theory and later VAE variants, so watching the entire recording would consume too much of your three-hour window.

These are the most useful sections of the course recording:

Priority	Approximate timestamp	Focus
First	**26:00–34:00**	Training objective; what the encoder and decoder do; reconstruction and regularization
Second	**54:00–59:00**	Reparameterization and the complete training diagram
Third	**63:00–69:00**	PyTorch architecture, loss, and inference
If the KL term is confusing	34:00–43:00	Gaussian KL and the encoder update
If squared error seems arbitrary	43:00–50:00	Decoder likelihood and its connection to reconstruction error
If tensor shapes are confusing	59:00–63:00	Convolutional and transposed-convolutional feature maps

The three priority segments total **about 19 minutes**, or around 15 minutes at 1.25× before pauses. These boundaries are approximate and based on inspected video frames.

For this homework, prioritize **the VAE recording and the metric sections of the linked Ignite tutorial**. The GAN, diffusion, Transformer, and multimodal recordings in the shared folder can wait.

Two problems in the supplied teaching material to watch for

The architecture slide appears to have a size mismatch. Around 66:00, the displayed encoder downsamples a 64×64 image five times, ending at 2×2. The displayed decoder upsamples only four times, ending at 32×32. Its last ordinary convolution preserves that size.

Before training, require:

Python

```
assert reconstructed_images.shape == input_images.shape
```

That check could save considerable debugging time.

The Ignite tutorial has a typo reversing the desired metric directions. The correct interpretation is:

> **Lower FID is better; higher Inception Score is conventionally better.**

The tutorial's closing note states the reverse, even though its preceding discussion describes the correct trends. pytorch-ignite.ai

3. Your three-hour work plan

This schedule assumes you can get a Python/GPU environment working promptly. Training should overlap report preparation.

Elapsed time	Task	What should be working when you move on
0–15 min	Check submission rules, GPU, imports, dataset download, and report template	Hardware identified; data location known
15–35 min	Watch the selected video segments and read the conceptual explanation below	You can explain the encoder, sampling, decoder, and two losses
35–75 min	Implement the loader and compact VAE; debug one batch	Matching image shapes and a finite training loss
75–85 min	Run and time pilot epochs; launch the baseline	A measured runtime estimate and saved checkpoint
85–125 min	Draft methods and hyperparameters; prepare evaluation while training runs	Report structure ready; metric pipeline checked
125–150 min	Save figures and calculate scores	Actual losses, generated images, and metrics
150–175 min	Finish the two-page report and clean the code	Complete, reproducible submission
175–180 min	Upload and verify both files	Correct submission confirmed

Decision checkpoint: if the first training update still does not work after approximately 75–90 minutes, revise the three-hour target. A functioning baseline should take priority over

adding complexity.

If training runs beyond the morning, you can complete the final results/report block later. That may still fit roughly three hours of **active work**, even when elapsed time is longer.

---

4. The VAE concepts you need to understand

Your MLIP experience is a useful starting point. You already know the general pattern: numerical input, a neural network, a loss, and optimization of its weights.

The main change is the goal. An MLIP predicts quantities such as energies and forces for supplied configurations. Here, you want a model that can **produce new images resembling the training distribution**.

Encoder, latent space, and decoder

A VAE has three central operations:

Component	What it does
Encoder	Reads an image and predicts a distribution over compact numerical representations
Latent sample	Draws one representation from that distribution
Decoder	Converts the representation into an image

The compact representation is the **latent vector**. Its entries are learned numerical coordinates; they do not automatically correspond to recognizable concepts such as lung size or pneumonia severity.

For a latent dimension of 32, the encoder produces:

- **32 means**.
- **32 log variances**.

Thus, it predicts 64 numbers describing a distribution from which a 32-number code can be sampled.

Reconstruction versus generation

This distinction is essential for both your implementation and report.

Reconstruction begins with a real image. You encode it and decode its representation. This tests whether the model can preserve information about an observed input.

Generation begins with a newly sampled latent vector. You send that directly to the decoder, without supplying a real image.

Your report needs examples of **fresh generation**. A reconstruction grid is a useful diagnostic, but it answers a different question.

What is the prior?

The **prior** is the distribution used to draw new latent codes. A common choice is independent normal coordinates with mean zero and variance one.

The encoder's distribution depends on the input image. The prior does not.

The training objective encourages those distributions to remain compatible, so that sampling a code from the prior is more likely to land in a region the decoder has learned to use. The Gaussian assumption concerns the latent codes; it does not require chest X-ray images themselves to follow a Gaussian distribution.

Why the reparameterization trick matters

For one image, define:

- j: the index of a latent coordinate.
- μ_j: the encoder's predicted mean for that coordinate.
- σ_j: its predicted standard deviation.
- $\ell_j = \log(\sigma_j^2)$: its predicted **log variance**; the superscript 2 means square, and $\log$ is the natural logarithm.
- ϵ_j: a random normal draw with mean zero and variance one.
- z_j: the sampled latent coordinate.

These quantities are dimensionless. Then:

$$\sigma_j = \exp(\ell_j/2), \qquad z_j = \mu_j + \sigma_j \epsilon_j.$$

Here, $\exp$ is the exponential function, the inverse of the natural logarithm.

In Python:

Python

```
std = torch.exp(0.5 * log_var)
z = mu + std * torch.randn_like(std)
```

The random draw supplies the noise; the remaining operations allow the reconstruction loss to propagate gradients back to the encoder. This is the **reparameterization trick**. The

factor `0.5` is necessary because you need the standard deviation, whereas exponentiating log variance gives the variance. arxiv.org

Why there are two loss terms

Let:

- L be the total loss.
- R be the reconstruction penalty.
- K be the Kullback–Leibler, or **KL**, divergence from the encoder distribution to the prior.
- β be the chosen weight controlling their relative importance.

The practical objective is:

$$L = R + \beta K.$$

Reconstruction penalty: “Does the output resemble the input?”

KL penalty: “How far does the encoded distribution depart from the distribution from which we want to sample new codes?”

For the usual diagonal Gaussian encoder, let d be the number of latent coordinates and let $\sum_{j=1}^{d}$ mean “add over all coordinates.” Using the means and log variances defined above:

$$K = \frac{1}{2} \sum_{j=1}^{d} \left[\mu_j^2 + \exp(\ell_j) - 1 - \ell_j \right].$$

With natural logarithms, KL is measured in **nats**, a dimensionless information unit. It is nonnegative. It becomes zero when all means are zero and all variances are one.

However, **minimizing KL alone is not the goal**: making the encoded distribution identical for every input can discard useful information. Reconstruction and KL must be considered together.

The loss convention I recommend for your first run

Your course slide uses:

Python

```
reconstruction = F.mse_loss(recon, images, reduction="mean")

kl = -0.5 * (
    1 + log_var - mu.square() - log_var.exp()
```

```
).sum(dim=1).mean()

loss = reconstruction + 0.001 * kl
```

Here, `F` is introduced with:

```
Python

import torch.nn.functional as F
```

This averages reconstruction error over pixels and the batch, while summing KL across latent coordinates and then averaging over the batch.

Keep those reductions consistent. At 64×64 grayscale resolution, changing reconstruction from a pixel mean to a pixel sum multiplies it by **4,096**. Leaving the KL coefficient unchanged would create a very different balance.

The coefficient `0.001` is a starting choice tied to this convention, not a universal VAE constant.

What is the ELBO?

The **evidence lower bound**, or ELBO, is a tractable lower bound on the log likelihood of the observed data. With a properly specified negative log-likelihood reconstruction term, minimizing reconstruction plus KL corresponds to maximizing the standard ELBO.

For tomorrow, understand why the two terms exist. The full derivation can follow after submission. Also, avoid automatically labeling a weighted mean-MSE loss as a numerical ELBO: its interpretation depends on likelihood scaling and reduction conventions.

---

5. A sensible baseline—and what its hyperparameters mean

I would start with the following configuration. These are **my recommendations**, not requirements from the handout.

Hyperparameter	Proposed starting value	Meaning and tradeoff
Image resolution	**64×64**	Smaller images train faster and simplify debugging, while losing detail
Image channels	**1, grayscale**	Appropriate for this baseline; convert to three channels separately for Inception evaluation

Hyperparameter	Proposed starting value	Meaning and tradeoff
Encoder channels	**32, 64, 128, 256**	Number of learned feature maps at each stage; more channels increase capacity and cost
Latent dimension	**32**	Number of coordinates in the sampled code
Batch size	**64 images**	Number of images used for one optimizer update
Optimizer	**Adam**	The rule used to update learned weights from gradients
Learning rate	**0.001**	Update scale; too high can destabilize learning, too low can waste time
KL weight	**0.001**	For the specific loss convention above
Epochs	**2 pilot, then continue to 30 total**	One epoch is one pass through the training set
Output activation	`tanh`	Produces values between −1 and 1; normalize training images to match
Random seed	One fixed, reported value	Helps reproduce initialization and sampling

The learning rate, KL weight, and latent coordinates have no standard physical units. Batch size is an image count; epochs are counts of passes through the dataset.

Implementation sequence

First, load and display images. The dataset has train, test, and validation folders with normal/pneumonia subfolders. For an unconditional VAE, those class labels can be ignored. Keep test images out of training, and count the actual extracted files. kaggle.com

Second, make the dimensions explicit. Let B be the number of images in the current batch. Your input shape should be:

```
Plain text
[B, 1, 64, 64]
```

Four downsampling stages can reduce the spatial size through 32, 16, 8, and 4 pixels per side. Your decoder must reverse all four stages.

The encoder means and log variances should each have shape:

```
Plain text
[B, 32]
```

Third, run one training update. The sequence is:

1. Clear old gradients.
2. Run the model.
3. Compute reconstruction, KL, and total loss.
4. Backpropagate.
5. Update the weights.

Check that losses are finite and the reconstructed image shape matches the input.

Fourth, run the pilot. Inspect reconstructions and fresh samples, measure complete-epoch time, and save a checkpoint. Continue that run to 30 total epochs if it behaves sensibly.

Fifth, collect the evidence. Save:

- Reconstruction, KL, and total-loss histories.
- A grid of newly generated images.
- Optionally, a real-image/reconstruction comparison.
- The trained checkpoint and configuration.

A large hyperparameter search is unnecessary for the scope described in the handout.

6. How long will training take?

I have not benchmarked this model on your hardware. The following are broad planning allowances for approximately 5,200 training images, a compact 64×64 convolutional VAE, batch size 64, working software, and local data.

Hardware	Initial training-only allowance for 50 epochs
Your RTX 4080 laptop, if still available and CUDA works	About 10–60 minutes
T4-class notebook GPU	About 20–120 minutes
CPU only	Potentially several hours; roughly 1–6+ hours is possible

These are not measured benchmarks or dependable upper bounds. Image decoding, storage speed, network architecture, and laptop power limits can substantially change the outcome. An A100 allocation could also work well, but account for queue and environment-setup time.

For the proposed **30 epochs**, steady-state training takes about 60% of the time for 50.

Replace that estimate with a measurement

Let:

- E be the number of training epochs.
- t_{epoch} be the measured time per epoch, in seconds.
- T_{train} be total training time, in seconds.

Then:

$$T_{\text{train}} \approx E\, t_{\text{epoch}}.$$

For example:

Measured time per epoch	30 epochs	50 epochs
45 seconds	22.5 minutes	37.5 minutes
3 minutes	90 minutes	150 minutes

Time a complete epoch with the real data loader after initial warm-up. This is much more useful than timing a single artificial batch.

Also budget for **evaluation, figures, and writing**. I would initially reserve roughly 15–45 minutes for metric setup/evaluation and 45–60 minutes for final figures, report completion, and submission, with some of that preparation overlapping training.

My deadline recommendation: aim for the regular deadline. Use the morning pilot to decide whether longer training fits. If the basic model is working by late morning, an unattended run followed by final evaluation later in the day is plausible. Check the extra-date policy before relying on it, and stop optional tuning at least two hours before the verified cutoff.

---

7. Inception Score and FID, explained plainly

“Inception” here refers to a pretrained image-classification network used as an evaluator. It is separate from your VAE.

Inception Score: confidence and category variety

Inception Score asks two questions:

1. Does the classifier make confident predictions for individual generated images?
2. Across the generated collection, does it predict a variety of categories?

It uses **generated images only**. Higher is conventionally preferred. It is not classification accuracy and is not a percentage. arxiv.org

Consider two failure cases:

- Every generated image receives the same confident category prediction: little category variety.
- Every generated image produces the same uncertain prediction: little confidence.

Both can produce a low score.

For chest X-rays, the limitation is substantial: the classifier’s ImageNet categories are not normal/pneumonia categories. Its confidence does not measure diagnostic correctness, and the score can miss repetition within a category or memorization. arxiv.org

FID: similarity between real and generated feature distributions

Fréchet Inception Distance uses **both real and generated images**.

The evaluator converts each image into a numerical feature vector. FID then compares:

- The **average feature vector** in each collection.
- The **covariance** of the features—how they vary and vary together.

Lower is better when the evaluation protocol is held fixed. A value of zero means the Gaussian feature summaries match; it does not prove that the full image distributions are identical. arxiv.org

An analogy to your research: two simulated systems can have matching averages and some fluctuation statistics while differing in other structural details. Similarly, matching image-feature statistics does not establish that every generated image is anatomically meaningful.

How the scores compare

Question	Inception Score	FID
Uses generated images?	Yes	Yes
Uses real reference images?	No	Yes
Preferred direction	Higher	Lower
Main comparison	Prediction confidence and category diversity	Real/generated feature means and covariances
Establishes clinical correctness?	No	No

Both estimates depend on finite samples. Their finite-sample bias does not disappear merely because you use the same number of images for each model. Report sample counts and avoid treating small differences as definitive evidence of superiority. arxiv.org

Practical evaluation rules for this assignment

Use the linked Ignite metric classes, with these checks:

- Evaluate **fresh samples from the latent prior**.
- Apply identical preprocessing to real and generated images.
- Convert the proposed VAE's output from `[-1, 1]` to `[0, 1]`, then repeat the grayscale channel three times.
- Use the preprocessing required by the chosen Inception weights.
- Accumulate the whole evaluation set and compute once; do not average separate per-batch FIDs. Torchvision 0.29 documentation

One implementation detail belongs in your report: current Ignite defaults use 1,000 classifier-output coordinates for FID, whereas a common FID implementation uses 2,048 pooled features. Record the package version and feature extractor. Numerical values from those configurations should not be treated as interchangeable. The guide explains this and provides the direct evaluation API sketch. PyTorch-Ignite v0.5.1 Documentation

8. What your two-page report should say

A compact, reproducible experiment is a suitable scope. Organize the report around these questions:

Section	What to include
Goal and data	Dataset, image counts/splits, resolution, grayscale conversion, normalization
Model and loss	Encoder/decoder structure, latent size, reconstruction loss, KL weight and reductions
Training	Optimizer, learning rate, batch size, completed epochs, seed, hardware, time
Results	Loss plot, generated-image grid, IS/FID table
Interpretation	What the model learned, visible weaknesses, and metric limitations

Use the course's template if supplied; otherwise, the official NeurIPS template is available. Follow the homework's two-page limit. Overleaf, Online LaTeX Editor

Useful questions to answer in your own words are:

- Did reconstruction improve?
- Did the KL term remain meaningful, or become nearly zero?
- Do new samples show recognizable chest structure?
- How much variation is visible between samples?
- Are reconstructions better than fresh generations?
- What can the scores support, given the feature extractor and sample count?

You do not need to claim excellent image quality for this to be an informative experiment. Explain the observed behavior accurately and connect it to the design choices.

Your first concrete checkpoint tomorrow is simple: detect the GPU, display one batch, and confirm that the model reconstructs the same image shape. Once those work, the timed pilot will give you a defensible training schedule.